

JavaScript

Violet

2026-04-29

Chapter 0

目 录

I	JS & TS 核心基础	
1.	JS & TS 概述与环境搭建	6
2.	基础语法与变量	7
3.	函数与作用域	8
4.	对象与数组	9
II	JavaScript 核心特性	
5.	原型与面向对象	11
6.	this 与执行上下文	12
7.	异步编程与事件循环	13
7.1.	JavaScript 异步模型概览	13
7.2.	事件循环机制	13
7.3.	Promise: 异步编程的核心	13
7.4.	async/await: 语法糖的威力	24
7.5.	高级异步模式	34
7.6.	定时器与异步调度	45
7.7.	实战示例	45
VI	Node.js 与 Bun	
8.	Node.js 基础与演进	50
8.1.	Node.js 架构	50
8.2.	版本演进	52
8.3.	模块系统	54
8.4.	全局对象	57
9.	Bun 现代运行时	60
9.1.	Bun 简介	60
9.2.	Bun vs Node.js	62
9.3.	Bun 特有 API	64
9.4.	迁移指南	68

10. 文件系统与路径 ·····	72
11. HTTP 服务器 ·····	73
12. Express/Koa 框架 ·····	74
13. Node.js/Bun 高级主题 ·····	75

VII

JavaScript 工程化

14. 包管理器 ·····	77
14.1. npm ·····	77
14.2. yarn ·····	84
14.3. pnpm ·····	87
14.4. Bun ·····	90
15. 构建工具 ·····	94
16. 代码质量 ·····	95
17. 测试框架 ·····	96
18. 部署与 CI/CD ·····	97



JS & TS 核心基础

1.	JS & TS 概述与环境搭建 ·····	6
2.	基础语法与变量 ·····	7
3.	函数与作用域 ·····	8
4.	对象与数组 ·····	9

Chapter 1

JS & TS 概述与环境搭建

Chapter 2

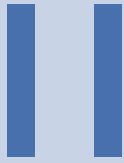
基础语法与变量

Chapter 3

函数与作用域

Chapter 4

对象与数组



JavaScript 核心特性

5. 原型与面向对象·····	11
6. this 与执行上下文·····	12
7. 异步编程与事件循环·····	13

Chapter 5

原型与面向对象

Chapter 6

this 与执行上下文

Chapter 7

异步编程与事件循环

1 JavaScript 异步模型概览

1.1 同步 vs 异步

1.2 为什么需要异步

1.3 异步编程的演进历程

2 事件循环机制

2.1 调用栈 (Call Stack)

2.2 任务队列 (Task Queue)

2.3 微任务与宏任务

2.4 事件循环执行流程

2.5 浏览器 vs Node.js 事件循环

3 Promise：异步编程的核心

Promise 是 JavaScript 异步编程的核心抽象，代表一个异步操作的最终完成（或失败）及其结果值。

3.1 Promise 基础概念

Promise 是一个对象，用于表示异步操作的最终完成（或失败）。

三种状态：

状态	说明	可否改变
pending（等待中）	初始状态，既不是成功也不是失败	✓

fulfilled (已成功)	操作成功完成	X
rejected (已失败)	操作失败	X

状态转换：

```
1 pending → fulfilled (resolve)
2 pending → rejected (reject)
```

基本语法：

```
1 const promise = new Promise((resolve, reject) => {
2   // 异步操作
3   setTimeout(() => {
4     const success = true;
5
6     if (success) {
7       resolve("操作成功"); // 状态变为 fulfilled
8     } else {
9       reject(new Error("操作失败")); // 状态变为 rejected
10    }
11  }, 1000);
12 });
13
14 promise
15 .then(result => {
16   console.log(result); // "操作成功"
17 })
18 .catch(error => {
19   console.error(error); // 如果失败则执行
20 });
```

Promise 的特点：

特点	说明
状态不可逆	一旦从 pending 变为 fulfilled/rejected，状态不再改变
链式调用	then/catch 返回新的 Promise，可以链式调用
微任务	Promise 回调作为微任务执行，优先级高于宏任务
错误冒泡	错误会沿着链式调用向下传递，直到被 catch 捕获
立即执行	Promise 构造函数中的代码会立即执行

3.2 Promise 状态机

Promise 的状态变化遵循严格的状态机规则。

状态转换图：

```
1      resolve()
2  pending -----> fulfilled
3  |
4  | then() 回调
```



示例：状态变化：

```

1  // 示例 1: 成功状态
2  const p1 = new Promise((resolve, reject) => {
3    console.log("Promise 创建"); // 立即执行
4    resolve("成功");
5  });
6
7  p1.then(value => {
8    console.log(value); // "成功"
9    return "新值";
10 }).then(value => {
11   console.log(value); // "新值"
12 });
13
14 // 示例 2: 失败状态
15 const p2 = new Promise((resolve, reject) => {
16   reject(new Error("失败"));
17 });
18
19 p2.catch(error => {
20   console.error(error.message); // "失败"
21 });
22
23 // 示例 3: 状态不可逆
24 const p3 = new Promise((resolve, reject) => {
25   resolve("第一次");
26   reject("第二次"); // 这行不会生效
27 });
28
29 p3.then(value => {
30   console.log(value); // "第一次"
31 });

```

Pending 状态的持久性：

```

1  // Promise 一直处于 pending 状态
2  const pendingPromise = new Promise(() => {
3    // 既不 resolve 也不 reject
4  });
5
6  pendingPromise.then(() => {
7    console.log("永远不会执行");
8  });

```

```
8  });
9
10 // 超时控制
11 const timeoutPromise = new Promise((_, reject) => {
12   setTimeout(() => {
13     reject(new Error("Timeout"));
14   }, 5000);
15 });
```

3.3 then/catch/finally

这三个方法是 Promise 的核心 API。

then 方法：

`then` 方法接受两个可选参数：成功回调和失败回调。

```
1 promise.then(
2   onFulfilled, // 可选, 当 Promise fulfilled 时调用
3   onRejected   // 可选, 当 Promise rejected 时调用
4 );
```

 JavaScript

基本用法：

```
1 const promise = fetchData();
2
3 // 方式 1: 两个参数
4 promise.then(
5   result => console.log("成功:", result),
6   error => console.error("失败:", error)
7 );
8
9 // 方式 2: then + catch (推荐)
10 promise
11   .then(result => console.log("成功:", result))
12   .catch(error => console.error("失败:", error));
```

 JavaScript

返回值：

```
1 // then 返回新的 Promise
2 promise
3   .then(result => {
4     console.log(result); // "原始值"
5     return "新值"; // 返回新值, 下一个 then 接收
6   })
7   .then(newResult => {
8     console.log(newResult); // "新值"
9     return Promise.resolve("Promise 值"); // 返回 Promise
10  })
11   .then(finalResult => {
12     console.log(finalResult); // "Promise 值"
13   });
```

 JavaScript

catch 方法：

`catch` 是 `then(null, onRejected)` 的语法糖。

```
1 // 这两种写法等价
2 promise.catch(error => console.error(error));
3 promise.then(null, error => console.error(error));
```

 JavaScript

错误捕获：

```
1 fetchData()
2   .then(result => {
3     // 如果这里抛出错误，会被后面的 catch 捕获
4     if (!result) {
5       throw new Error("数据为空");
6     }
7     return process(result);
8   })
9   .then(processed => {
10    return save(processed);
11  })
12  .catch(error => {
13    // 捕获上面任何一步的错误
14    console.error("错误:", error.message);
15  });
```

 JavaScript

NOTE

`catch` 会捕获前面所有步骤的错误，包括 `then` 回调中抛出的错误。

`finally` 方法：

`finally` 无论 Promise 成功还是失败都会执行，常用于清理工作。

```
1 showLoading();
2
3 fetchData()
4   .then(result => {
5     display(result);
6   })
7   .catch(error => {
8     showError(error);
9   })
10  .finally(() => {
11    // 无论成功还是失败，都会执行
12    hideLoading();
13  });
```

 JavaScript

特点：

特点	说明
必定执行	无论 fulfilled 还是 rejected 都会执行
无参数	不接收任何参数
不改变结果	返回值不影响 Promise 的最终结果
透传	继续传递原始的 resolved/rejected 值

示例:

```
1 Promise.resolve("成功")
2   .finally(() => {
3     console.log("清理工作");
4     return "被忽略"; // 这个返回值会被忽略
5   })
6   .then(value => {
7     console.log(value); // "成功" (原始值)
8   });
9
10 Promise.reject(new Error("失败"))
11   .finally(() => {
12     console.log("清理工作");
13   })
14   .catch(error => {
15     console.error(error.message); // "失败" (原始错误)
16   });
```

3.4 Promise 链式调用

Promise 的强大之处在于链式调用，可以避免回调地狱。

基本链式调用:

```
1 // 串行执行: 每一步依赖上一步的结果
2 fetchUser(userId)
3   .then(user => {
4     return fetchPosts(user.id); // 返回 Promise
5   })
6   .then(posts => {
7     return fetchComments(posts[0].id); // 返回 Promise
8   })
9   .then(comments => {
10    console.log(comments);
11  })
12  .catch(error => {
13    console.error("任何一步出错都会到这里", error);
14  });
```

返回值类型:

```
1 promise
2   .then(result => {
3     // 情况 1: 返回普通值
4     return "字符串";
5     // 下一个 then 接收: "字符串"
6   })
7   .then(value => {
8     // 情况 2: 返回 Promise
9     return Promise.resolve(42);
10    // 下一个 then 接收: 42
11  });
```

```
12 .then(value => {
13     // 情况 3: 不返回 (返回 undefined)
14     console.log(value); // 42
15     // 下一个 then 接收: undefined
16 })
17 .then(value => {
18     console.log(value); // undefined
19 });
```

链式调用的错误传播:

```
1 step1()
2 .then(result1 => step2(result1))
3 .then(result2 => step3(result2)) // 如果 step2 失败, 跳过这里
4 .then(result3 => step4(result3)) // 如果 step3 失败, 跳过这里
5 .catch(error => {
6     // 捕获 step1、step2、step3、step4 中任何一个的错误
7     console.error(error);
8 });
```

 JavaScript

局部错误处理:

```
1 step1()
2 .then(result1 => {
3     return step2(result1).catch(error => {
4         // 只捕获 step2 的错误
5         console.warn("step2 失败, 使用默认值");
6         return defaultValue;
7     });
8 })
9 .then(result2 => {
10     // 即使 step2 失败, 这里仍会执行 (使用默认值)
11     return step3(result2);
12 })
13 .catch(error => {
14     // 捕获 step1 或 step3 的错误
15     console.error(error);
16 });
```

 JavaScript

3.5 Promise 静态方法

Promise 提供了多个静态方法来处理多个 Promise。

Promise.all:

等待所有 Promise 都成功, 返回结果数组; 如果有一个失败, 立即拒绝。

```
1 const promise1 = Promise.resolve(3);
2 const promise2 = 42; // 非 Promise 值会被包装
3 const promise3 = new Promise(resolve => {
4     setTimeout(resolve, 100, "foo");
5 });
6
```

 JavaScript

```
7 Promise.all([promise1, promise2, promise3])
8   .then(values => {
9     console.log(values); // [3, 42, "foo"]
10  })
11  .catch(error => {
12    // 如果任何一个 Promise reject, 立即执行
13    console.error(error);
14  });
```

失败快速:

```
1 const p1 = Promise.resolve(1);
2 const p2 = Promise.reject(new Error("失败"));
3 const p3 = Promise.resolve(3);
4
5 Promise.all([p1, p2, p3])
6   .then(values => {
7     console.log(values); // 不会执行
8   })
9   .catch(error => {
10    console.error(error.message); // "失败"
11    // 注意: p3 可能还在执行, 但结果被忽略
12  });
```

 JavaScript

应用场景:

```
1 // 并行加载多个资源
2 async function loadResources() {
3   const [users, posts, comments] = await Promise.all([
4     fetch("/api/users").then(r => r.json()),
5     fetch("/api/posts").then(r => r.json()),
6     fetch("/api/comments").then(r => r.json())
7   ]);
8
9   return { users, posts, comments };
10 }
```

 JavaScript

Promise.race:

返回第一个 settled (fulfilled 或 rejected) 的 Promise 的结果。

```
1 const p1 = new Promise(resolve => {
2   setTimeout(resolve, 500, "first");
3 });
4
5 const p2 = new Promise(resolve => {
6   setTimeout(resolve, 100, "second");
7 });
8
9 Promise.race([p1, p2])
10  .then(value => {
11    console.log(value); // "second" (更快的那个)
12  });
```

 JavaScript

超时控制:

```
1 function fetchWithTimeout(url, timeout) {⚡ JavaScript  
2   const fetchPromise = fetch(url);  
3   const timeoutPromise = new Promise( (_, reject) => {  
4     setTimeout(() => reject(new Error("Request timeout")), timeout);  
5   });  
6  
7   return Promise.race([fetchPromise, timeoutPromise]);  
8 }  
9  
10 // 使用  
11 fetchWithTimeout("https://api.example.com/data", 5000)  
12   .then(response => response.json())  
13   .catch(error => {  
14     console.error(error.message); // "Request timeout" 或网络错误  
15   });
```

Promise.allSettled:

等待所有 Promise 都 settled，返回每个 Promise 的状态和结果。

```
1 const p1 = Promise.resolve(1);⚡ JavaScript  
2 const p2 = Promise.reject(new Error("失败"));  
3 const p3 = Promise.resolve(3);  
4  
5 Promise.allSettled([p1, p2, p3])  
6   .then(results => {  
7     console.log(results);  
8     // [  
9     //   { status: "fulfilled", value: 1 },  
10    //   { status: "rejected", reason: Error: 失败 },  
11    //   { status: "fulfilled", value: 3 }  
12    // ]  
13  });
```

处理部分失败:

```
1 async function loadAllResources(urls) {⚡ JavaScript  
2   const promises = urls.map(url =>  
3     fetch(url).then(r => r.json()).catch(error => ({ error })))  
4   );  
5  
6   const results = await Promise.allSettled(promises);  
7  
8   const successful = results  
9     .filter(r => r.status === "fulfilled")  
10    .map(r => r.value);  
11  
12   const failed = results  
13     .filter(r => r.status === "rejected")  
14     .map(r => r.reason);  
15  
16   console.log("成功:", successful.length);  
17   console.log("失败:", failed.length);  
18 }
```

```
19   return { successful, failed };
20 }
```

Promise.any:

返回第一个 fulfilled 的 Promise 的结果；如果全部 rejected，返回 AggregateError。

```
1  const p1 = Promise.reject(new Error("失败 1"));
2  const p2 = Promise.resolve("成功");
3  const p3 = Promise.reject(new Error("失败 2"));
4
5  Promise.any([p1, p2, p3])
6    .then(value => {
7      console.log(value); // "成功"
8    })
9    .catch(error => {
10     // 只有全部失败才会到这里
11     console.error(error); // AggregateError
12   });
```

 JavaScript

全部失败:

```
1  const p1 = Promise.reject(new Error("失败 1"));
2  const p2 = Promise.reject(new Error("失败 2"));
3
4  Promise.any([p1, p2])
5    .then(value => {
6      console.log(value); // 不会执行
7    })
8    .catch(error => {
9      console.error(error.name); // "AggregateError"
10     console.error(error.errors); // [Error: 失败 1, Error: 失败 2]
11   });
```

 JavaScript

应用场景：多源数据获取：

```
1  // 从多个 CDN 获取数据，使用最快的成功响应
2  const cdns = [
3    fetch("https://cdn1.example.com/data.json"),
4    fetch("https://cdn2.example.com/data.json"),
5    fetch("https://cdn3.example.com/data.json")
6  ];
7
8  Promise.any(cdns)
9    .then(response => response.json())
10   .then(data => {
11     console.log("从最快的 CDN 获取数据:", data);
12   })
13   .catch(error => {
14     console.error("所有 CDN 都失败:", error.errors);
15   });
```

 JavaScript

对比总结：

方法	成功条件	失败条件	返回值
Promise.all	全部 fulfilled	任一 rejected	结果数组 / 第一个错误
Promise.race	第一个 settled	第一个 rejected	第一个结果 / 错误
Promise.allSettled	全部 settled	永不失败	状态数组
Promise.any	任一 fulfilled	全部 rejected	第一个成功 / AggregateError

3.6 错误处理最佳实践

良好的错误处理是 Promise 使用的关键。

始终添加 catch:

```
1 // ❌ 错误做法: 未捕获的错误
2 fetchData().then(result => {
3   console.log(result);
4 });
5
6 // ✅ 正确做法: 总是添加 catch
7 fetchData()
8   .then(result => {
9     console.log(result);
10  })
11  .catch(error => {
12    console.error("Error:", error);
13  });
```

⚡ JavaScript

避免吞掉错误:

```
1 // ❌ 错误做法: 吞掉错误
2 promise.catch(error => {
3   // 什么都不做
4 });
5
6 // ✅ 正确做法: 记录或重新抛出
7 promise.catch(error => {
8   console.error(error);
9   // 或者重新抛出, 让上层处理
10  throw error;
11 });
```

⚡ JavaScript

统一的错误处理:

```
1 // 全局未处理的 Promise rejection
2 window.addEventListener("unhandledrejection", event => {
3   console.error("Unhandled rejection:", event.reason);
4   event.preventDefault(); // 防止控制台警告
5 });
6
7 // Node.js
8 process.on("unhandledRejection", (reason, promise) => {
9   console.error("Unhandled Rejection at:", promise, "reason:", reason);
```

⚡ JavaScript

```
10 });
```

async/await 中的错误处理:

```
1 // 方式 1: try-catch
2 async function loadData() {
3   try {
4     const response = await fetch("/api/data");
5     const data = await response.json();
6     return data;
7   } catch (error) {
8     console.error("Failed to load data:", error);
9     throw error; // 重新抛出或返回默认值
10  }
11 }
12
13 // 方式 2: .catch()
14 async function loadData() {
15   return fetch("/api/data")
16     .then(response => response.json())
17     .catch(error => {
18       console.error("Failed to load data:", error);
19       return null; // 返回默认值
20     });
21 }
```

4 async/await: 语法糖的威力

`async/await` 是基于 Promise 的语法糖, 让异步代码看起来像同步代码, 提高可读性和可维护性。

4.1 基本语法

`async` 关键字声明一个异步函数, `await` 关键字等待 Promise 完成。

async 函数

```
1 // 声明 async 函数
2 async function fetchData() {
3   return "data";
4 }
5
6 // async 函数总是返回 Promise
7 fetchData().then(data => {
8   console.log(data); // "data"
9 });
10
11 // 等价于
12 function fetchData() {
13   return Promise.resolve("data");
14 }
```


特点:

特点	说明
返回值	自动包装为 Promise
非 async 函数调用	可以调用 async 函数
异常处理	抛出的异常会被 reject
兼容性	ES2017+

await 表达式

`await` 只能在 `async` 函数内部使用，用于等待 Promise 完成。

```
1  async function loadData() {
2    // 等待 Promise 完成
3    const response = await fetch("/api/data");
4
5    // 继续等待 JSON 解析
6    const data = await response.json();
7
8    return data;
9  }
10
11 // 使用
12 loadData().then(data => {
13   console.log(data);
14 });
```

JavaScript

执行流程:

```
1  async function example() {
2    console.log("1. 开始");
3
4    const result = await someAsyncOperation();
5    console.log("2. 结果:", result);
6
7    console.log("3. 结束");
8  }
9
10 // 输出顺序:
11 // 1. 开始
12 // (等待 someAsyncOperation 完成)
13 // 2. 结果: xxx
14 // 3. 结束
```

JavaScript

对比: Promise vs async/await

```
1  // Promise 链式调用
2  function loadUserData(userId) {
3    return fetchUser(userId)
4      .then(user => fetchPosts(user.id))
5      .then(posts => fetchComments(posts[0].id))
```

JavaScript

```
6     .then(comments => {
7         return { user, posts, comments };
8     })
9     .catch(error => {
10         console.error(error);
11     });
12 }
13
14 // async/await (更清晰)
15 async function loadUserData(userId) {
16     try {
17         const user = await fetchUser(userId);
18         const posts = await fetchPosts(user.id);
19         const comments = await fetchComments(posts[0].id);
20
21         return { user, posts, comments };
22     } catch (error) {
23         console.error(error);
24     }
25 }
```

4.2 错误处理

async/await 提供了更直观的错误处理方式。

try-catch 方式

```
1  async function loadData() {
2      try {
3          const response = await fetch("/api/data");
4
5          if (!response.ok) {
6              throw new Error(`HTTP error! status: ${response.status}`);
7          }
8
9          const data = await response.json();
10         return data;
11
12     } catch (error) {
13         console.error("Failed to load data:", error.message);
14
15         // 可以选择:
16         // 1. 重新抛出
17         throw error;
18
19         // 2. 返回默认值
20         // return defaultValue;
21
22         // 3. 返回 null
23         // return null;
24     }
25 }
```

多个 await 的错误处理

```
1  async function loadAllData() {
2      try {
3          const users = await fetchUsers();
4          const posts = await fetchPosts();
5          const comments = await fetchComments();
6
7          return { users, posts, comments };
8
9      } catch (error) {
10         // 任何一个 await 失败都会到这里
11         console.error("Failed to load data:", error);
12         throw error;
13     }
14 }
```

NOTE

如果第一个 await 失败，后面的 await 不会执行。

独立的错误处理

```
1  async function loadAllData() {
2      let users, posts, comments;
3
4      // 每个请求独立处理错误
5      try {
6          users = await fetchUsers();
7      } catch (error) {
8          console.warn("Failed to load users:", error);
9          users = [];
10     }
11
12     try {
13         posts = await fetchPosts();
14     } catch (error) {
15         console.warn("Failed to load posts:", error);
16         posts = [];
17     }
18
19     try {
20         comments = await fetchComments();
21     } catch (error) {
22         console.warn("Failed to load comments:", error);
23         comments = [];
24     }
25
26     return { users, posts, comments };
27 }
```

Promise.catch 混合使用

```
1  async function loadData() {
```

```
2 // 方式 1: await + try-catch
3 try {
4   const data = await fetchData();
5   return data;
6 } catch (error) {
7   console.error(error);
8 }
9
10 // 方式 2: await + .catch()
11 const data = await fetchData().catch(error => {
12   console.error(error);
13   return null;
14 });
15
16 return data;
17 }
```

4.3 并行执行 vs 串行执行

理解并行和串行的区别对于性能优化至关重要。

串行执行（不推荐）

```
1 // ❌ 串行执行: 总耗时 = 1s + 1s + 1s = 3s
2 async function loadSerially() {
3   const users = await fetchUsers(); // 1s
4   const posts = await fetchPosts(); // 1s
5   const comments = await fetchComments(); // 1s
6
7   return { users, posts, comments };
8 }
```

执行时间线:

```
1 0s -----> 1s -----> 2s -----> 3s
2 |fetchUsers|fetchPosts|fetchComments|
```

并行执行（推荐）

```
1 // ✅ 并行执行: 总耗时 = max(1s, 1s, 1s) = 1s
2 async function loadInParallel() {
3   const [users, posts, comments] = await Promise.all([
4     fetchUsers(),
5     fetchPosts(),
6     fetchComments()
7   ]);
8
9   return { users, posts, comments };
10 }
```

执行时间线:

```
1 0s -----> 1s
```

```
2 | fetchUsers      |
3 | fetchPosts      |
4 | fetchComments   |
```

部分并行

当某些请求依赖其他请求的结果时，可以部分并行。

```
1  async function loadUserData(userId) { ⚡ JavaScript
2    // 第一步：获取用户（必须串行）
3    const user = await fetchUser(userId);
4
5    // 第二步：并行获取用户的帖子和评论
6    const [posts, comments] = await Promise.all([
7      fetchPosts(user.id),
8      fetchComments(user.id)
9    ]);
10
11   return { user, posts, comments };
12 }
```

执行时间线：

```
1  0s -----> 1s -----> 2s
2  | fetchUser | fetchPosts      |
3  |           | fetchComments   |
```

性能对比

```
1  // 测试性能差异 ⚡ JavaScript
2  async function testPerformance() {
3    console.time("Serial");
4    await loadSerially();
5    console.timeEnd("Serial"); // ~3000ms
6
7    console.time("Parallel");
8    await loadInParallel();
9    console.timeEnd("Parallel"); // ~1000ms
10 }
```

TIP

当多个异步操作互不依赖时，总是使用并行执行以提高性能。

4.4 常见陷阱与解决方案

async/await 虽然简化了异步编程，但也有一些常见的陷阱需要注意。

陷阱 1：忘记 await

```
1  // ❌ 错误：忘记 await ⚡ JavaScript
2  async function loadData() {
3    const data = fetchData(); // 返回 Promise，不是实际数据
```

```
4 console.log(data); // Promise { <pending> }
5 }
6
7 // ✅ 正确: 添加 await
8 async function loadData() {
9   const data = await fetchData();
10  console.log(data); // 实际数据
11 }
```

❗ 陷阱 2: 在循环中串行执行

```
1 // ❌ 错误: 串行执行, 性能差 ⚡ JavaScript
2 async function loadUsers(userIds) {
3   const users = [];
4   for (const id of userIds) {
5     const user = await fetchUser(id); // 每次等待
6     users.push(user);
7   }
8   return users;
9 }
10
11 // ✅ 正确: 并行执行
12 async function loadUsers(userIds) {
13   const promises = userIds.map(id => fetchUser(id));
14   return await Promise.all(promises);
15 }
```

性能对比:

```
1 // 假设有 100 个用户, 每个请求 100ms ⚡ JavaScript
2 // 串行: 100 * 100ms = 10s
3 // 并行: max(100ms) = 100ms
```

❗ 陷阱 3: 未处理的错误

```
1 // ❌ 错误: 未捕获的错误 ⚡ JavaScript
2 async function loadData() {
3   const data = await fetchData(); // 如果失败, 错误未被捕获
4   return data;
5 }
6
7 // ✅ 正确: 添加错误处理
8 async function loadData() {
9   try {
10     const data = await fetchData();
11     return data;
12   } catch (error) {
13     console.error(error);
14     return null;
15   }
16 }
```

陷阱 4：顶层 await 的限制

```
1 // ✗ 错误：在非 async 函数中使用 await ⚡ JavaScript
2 function loadData() {
3   const data = await fetchData(); // SyntaxError
4 }
5
6 // ✔ 正确 1：在 async 函数中
7 async function loadData() {
8   const data = await fetchData();
9 }
10
11 // ✔ 正确 2：顶层 await (ES Module)
12 // 只在 ES Module 中支持
13 const data = await fetchData();
```

NOTE

顶层 await 仅在 ES Module 中支持，CommonJS 不支持。

陷阱 5：过度使用 async/await

```
1 // ✗ 不必要：简单的 Promise 链 ⚡ JavaScript
2 async function loadData() {
3   const response = await fetch("/api/data");
4   const data = await response.json();
5   return data;
6 }
7
8 // ✔ 更简洁：直接返回 Promise
9 function loadData() {
10   return fetch("/api/data")
11     .then(response => response.json());
12 }
```

TIP

如果函数只是简单地转发 Promise，不需要使用 async/await。

陷阱 6：错误堆栈丢失

```
1 // ✗ 可能丢失堆栈信息 ⚡ JavaScript
2 async function loadData() {
3   try {
4     await fetchData();
5   } catch (error) {
6     throw new Error("Failed to load"); // 原始堆栈丢失
7   }
8 }
9
10 // ✔ 保留原始错误
11 async function loadData() {
12   try {
13     await fetchData();
```

```

14   } catch (error) {
15     error.message = "Failed to load: " + error.message;
16     throw error; // 保留原始堆栈
17   }
18 }

```

最佳实践总结

实践	说明	示例
总是 await	确保获取实际值	const data = await fetchData()
并行执行	互不依赖的请求并行	Promise.all
错误处理	使用 try-catch	try { await ... } catch {}
避免嵌套	扁平化异步代码	减少 then 链
简洁返回	简单场景直接返回 Promise	return fetch()
保留堆栈	不要创建新错误	修改原错误消息

完整示例

```

1  // 真实的 API 请求封装
2  class ApiService {
3    constructor(baseUrl) {
4      this.baseUrl = baseUrl;
5    }
6
7    async request(endpoint, options = {}) {
8      const url = `${this.baseUrl}${endpoint}`;
9
10     try {
11       const response = await fetch(url, {
12         headers: {
13           "Content-Type": "application/json",
14           ...options.headers
15         },
16         ...options
17       });
18
19       if (!response.ok) {
20         throw new Error(`HTTP ${response.status}: ${response.statusText}`);
21       }
22
23       return await response.json();
24
25     } catch (error) {
26       console.error(`Request failed: ${endpoint}`, error);
27       throw error;
28     }
29   }
30
31   async getUsers() {

```



```
32     return this.request("/users");
33 }
34
35 async getUser(id) {
36     return this.request(`/users/${id}`);
37 }
38
39 async createUser(userData) {
40     return this.request("/users", {
41         method: "POST",
42         body: JSON.stringify(userData)
43     });
44 }
45
46 async loadDashboard(userId) {
47     // 第一步：获取用户
48     const user = await this.getUser(userId);
49
50     // 第二步：并行获取相关数据
51     const [posts, comments, stats] = await Promise.all([
52         this.request(`/users/${userId}/posts`),
53         this.request(`/users/${userId}/comments`),
54         this.request(`/users/${userId}/stats`)
55     ]);
56
57     return {
58         user,
59         posts,
60         comments,
61         stats
62     };
63 }
64 }
65
66 // 使用
67 const api = new ApiService("https://api.example.com");
68
69 async function main() {
70     try {
71         const dashboard = await api.loadDashboard(123);
72         console.log("Dashboard loaded:", dashboard);
73     } catch (error) {
74         console.error("Failed to load dashboard:", error);
75     }
76 }
77
78 main();
```

本节详细介绍了 `async/await` 的基本语法、错误处理、并行执行以及常见陷阱。`async/await` 让异步代码更加清晰易读，是现代 JavaScript 异步编程的首选方式。

5 高级异步模式

在实际开发中，除了基础的 Promise 和 async/await，还需要掌握一些高级的异步编程模式来解决复杂场景。

5.1 回调地狱与解决方案

回调地狱（Callback Hell）是早期 JavaScript 异步编程的主要问题。

什么是回调地狱

```
1 // ❌ 回调地狱：嵌套层级深，难以维护
2 getUser(userId, function(user) {
3   getPosts(user.id, function(posts) {
4     getComments(posts[0].id, function(comments) {
5       getAuthor(comments[0].authorId, function(author) {
6         console.log(author);
7       });
8     });
9   });
10 });
```

问题：

问题	说明
嵌套过深	代码向右延伸，难以阅读
错误处理困难	每层都需要单独处理错误
控制流复杂	难以实现并行、竞争等逻辑
调试困难	堆栈跟踪不清晰

解决方案 1: Promise 链式调用

```
1 // ✅ Promise 链式调用
2 getUser(userId)
3   .then(user => getPosts(user.id))
4   .then(posts => getComments(posts[0].id))
5   .then(comments => getAuthor(comments[0].authorId))
6   .then(author => {
7     console.log(author);
8   })
9   .catch(error => {
10    console.error(error);
11  });
```

解决方案 2: async/await

```
1 // ✅ async/await: 最清晰的写法
2 async function loadAuthorInfo(userId) {
3   try {
4     const user = await getUser(userId);
```

```
5     const posts = await getPosts(user.id);
6     const comments = await getComments(posts[0].id);
7     const author = await getAuthor(comments[0].authorId);
8
9     console.log(author);
10    return author;
11  } catch (error) {
12    console.error(error);
13  }
14 }
```

解决方案 3: 命名函数

```
1  //  提取为命名函数 JavaScript
2  function handleUser(user) {
3    return getPosts(user.id);
4  }
5
6  function handlePosts(posts) {
7    return getComments(posts[0].id);
8  }
9
10 function handleComments(comments) {
11   return getAuthor(comments[0].authorId);
12 }
13
14 getUser(userId)
15   .then(handleUser)
16   .then(handlePosts)
17   .then(handleComments)
18   .then(author => console.log(author))
19   .catch(console.error);
```

5.2 发布-订阅模式

发布-订阅 (Pub/Sub) 模式是一种事件驱动的异步模式，解耦生产者和消费者。

基本实现

```
1  class EventEmitter { JavaScript
2    constructor() {
3      this.events = {};
4    }
5
6    // 订阅事件
7    on(event, callback) {
8      if (!this.events[event]) {
9        this.events[event] = [];
10     }
11     this.events[event].push(callback);
12
13     // 返回取消订阅函数
```

```
14     return () => {
15         this.events[event] = this.events[event].filter(cb => cb !== callback);
16     };
17 }
18
19 // 发布事件
20 emit(event, ...args) {
21     if (this.events[event]) {
22         this.events[event].forEach(callback => {
23             callback(...args);
24         });
25     }
26 }
27
28 // 只订阅一次
29 once(event, callback) {
30     const unsubscribe = this.on(event, (...args) => {
31         callback(...args);
32         unsubscribe(); // 自动取消订阅
33     });
34 }
35
36 // 取消所有订阅
37 off(event) {
38     delete this.events[event];
39 }
40 }
```

使用示例

```
1  const emitter = new EventEmitter();
2
3  // 订阅事件
4  const unsubscribe = emitter.on("data", (data) => {
5      console.log("收到数据:", data);
6  });
7
8  // 发布事件
9  emitter.emit("data", { id: 1, value: "test" });
10 // 输出: 收到数据: { id: 1, value: "test" }
11
12 // 取消订阅
13 unsubscribe();
14 emitter.emit("data", { id: 2, value: "test2" });
15 // 不会输出, 因为已取消订阅
```

 JavaScript

异步事件处理

```
1  class AsyncEventEmitter extends EventEmitter {
2      // 异步发布, 等待所有处理器完成
3      async emitAsync(event, ...args) {
4          if (this.events[event]) {
```

 JavaScript

```
5     const promises = this.events[event].map(callback => {
6       try {
7         return Promise.resolve(callback(...args));
8       } catch (error) {
9         return Promise.reject(error);
10      }
11    });
12
13    return Promise.allSettled(promises);
14  }
15 }
16 }
17
18 // 使用
19 const asyncEmitter = new AsyncEventEmitter();
20
21 asyncEmitter.on("request", async (url) => {
22   const response = await fetch(url);
23   return response.json();
24 });
25
26 asyncEmitter.on("request", async (url) => {
27   console.log("Logging request to:", url);
28 });
29
30 // 发布并等待所有处理器完成
31 const results = await asyncEmitter.emitAsync("request", "https://api.example.com");
32 console.log(results);
33 // [
34 //   { status: "fulfilled", value: {...} },
35 //   { status: "fulfilled", value: undefined }
36 // ]
```

应用场景

场景	说明	示例
事件驱动架构	组件间通信	UI 事件、消息队列
插件系统	扩展点机制	Webpack 插件、Babel 插件
状态管理	状态变化通知	Redux、Vuex
实时通信	WebSocket 消息	聊天应用、实时通知
日志系统	多目标日志	控制台、文件、远程服务器

5.3 生产者-消费者模式

生产者-消费者模式用于处理异步数据流，解耦数据生产和消费。

基于 Promise 的实现

```
1 class TaskQueue {
```

 JavaScript

```
2   constructor(concurrency = 1) {
3     this.queue = [];
4     this.concurrency = concurrency;
5     this.running = 0;
6   }
7
8   // 添加任务
9   add(task) {
10    return new Promise((resolve, reject) => {
11      this.queue.push({
12        task,
13        resolve,
14        reject
15      });
16
17      this.process();
18    });
19  }
20
21  // 处理任务
22  async process() {
23    if (this.running >= this.concurrency || this.queue.length === 0) {
24      return;
25    }
26
27    this.running++;
28    const { task, resolve, reject } = this.queue.shift();
29
30    try {
31      const result = await task();
32      resolve(result);
33    } catch (error) {
34      reject(error);
35    } finally {
36      this.running--;
37      this.process(); // 处理下一个任务
38    }
39  }
40 }
```

使用：

```
1  const queue = new TaskQueue(3); // 最多 3 个并发
2
3  // 添加 10 个任务
4  for (let i = 0; i < 10; i++) {
5    queue.add(() => {
6      console.log(`Task ${i} started`);
7      return new Promise(resolve => {
8        setTimeout(() => {
9          console.log(`Task ${i} completed`);
10         resolve(i);
11       }, 1000);
12     });
13   }
```

 JavaScript

```
12     });
13   }).then(result => {
14     console.log(`Result: ${result}`);
15   });
16 }
17
18 // 输出:
19 // Task 0 started
20 // Task 1 started
21 // Task 2 started
22 // (1秒后)
23 // Task 0 completed
24 // Result: 0
25 // Task 3 started
26 // ...
```

基于生成器的实现

```
1  function* producer() {
2    let i = 0;
3    while (true) {
4      yield i++;
5    }
6  }
7
8  function* consumer(producer) {
9    while (true) {
10     const value = producer.next().value;
11     console.log("Consumed:", value);
12     yield value;
13   }
14 }
15
16 // 使用
17 const prod = producer();
18 const cons = consumer(prod);
19
20 cons.next(); // Consumed: 0
21 cons.next(); // Consumed: 1
22 cons.next(); // Consumed: 2
```

 JavaScript

基于 AsyncIterator 的实现

```
1  class AsyncQueue {
2    constructor() {
3      this.items = [];
4      this.resolvers = [];
5    }
6
7    // 生产者: 添加数据
8    push(item) {
9      if (this.resolvers.length > 0) {
```

 JavaScript

```
10     const resolve = this.resolvers.shift();
11     resolve({ value: item, done: false });
12   } else {
13     this.items.push(item);
14   }
15 }
16
17 // 消费者：获取数据
18 pop() {
19   if (this.items.length > 0) {
20     return Promise.resolve({
21       value: this.items.shift(),
22       done: false
23     });
24   }
25
26   return new Promise(resolve => {
27     this.resolvers.push(resolve);
28   });
29 }
30
31 // 实现 AsyncIterator
32 [Symbol.asyncIterator]() {
33   return {
34     next: () => this.pop()
35   };
36 }
37 }
```

使用：

```
1  const queue = new AsyncQueue();
2
3  // 消费者：异步迭代
4  (async () => {
5    for await (const item of queue) {
6      console.log("Processed:", item);
7    }
8  })();
9
10 // 生产者：添加数据
11 setTimeout(() => queue.push(1), 1000);
12 setTimeout(() => queue.push(2), 2000);
13 setTimeout(() => queue.push(3), 3000);
14
15 // 输出：
16 // (1秒后) Processed: 1
17 // (2秒后) Processed: 2
18 // (3秒后) Processed: 3
```

 JavaScript

应用场景区

场景	说明	示例
----	----	----

任务队列	限制并发数	图片上传、API 请求
数据流处理	背压控制	文件读写、网络流
消息队列	异步消息处理	RabbitMQ、Kafka
批处理	批量处理数据	数据库批量插入
工作线程池	CPU 密集型任务	图像处理、计算

5.4 异步迭代器与生成器

异步迭代器和生成器提供了处理异步数据流的优雅方式。

生成器基础

生成器函数可以暂停和恢复执行。

```
1 function* numberGenerator() {  
2   yield 1;  
3   yield 2;  
4   yield 3;  
5 }  
6  
7 const gen = numberGenerator();  
8  
9 console.log(gen.next()); // { value: 1, done: false }  
10 console.log(gen.next()); // { value: 2, done: false }  
11 console.log(gen.next()); // { value: 3, done: false }  
12 console.log(gen.next()); // { value: undefined, done: true }
```

异步生成器

异步生成器可以 yield Promise。

```
1 async function* asyncNumberGenerator() {  
2   yield await Promise.resolve(1);  
3   yield await Promise.resolve(2);  
4   yield await Promise.resolve(3);  
5 }  
6  
7 // 使用 for-await-of  
8 (async () => {  
9   for await (const num of asyncNumberGenerator()) {  
10    console.log(num); // 1, 2, 3  
11   }  
12 })();
```

实战：分页数据加载

```
1 async function* fetchPages(baseUrl) {  
2   let page = 1;  
3   let hasMore = true;  
4  
5   while (hasMore) {
```

```
6     const response = await fetch(`${baseUrl}?page=${page}`);
7     const data = await response.json();
8
9     for (const item of data.items) {
10         yield item;
11     }
12
13     hasMore = data.hasMore;
14     page++;
15 }
16 }
17
18 // 使用：像同步代码一样处理异步数据流
19 (async () => {
20     for await (const item of fetchPages("https://api.example.com/items")) {
21         console.log(item);
22         // 可以在这里处理每个 item
23         // 不需要关心分页逻辑
24     }
25 })();
```

实战：无限滚动

```
1  async function* infiniteScroll(fetchFn) { ⚡ JavaScript
2      let cursor = null;
3
4      while (true) {
5          const { items, nextCursor } = await fetchFn(cursor);
6
7          for (const item of items) {
8              yield item;
9          }
10
11          if (!nextCursor) {
12              break;
13          }
14
15          cursor = nextCursor;
16      }
17 }
18
19 // 使用
20 const fetchPosts = async (cursor) => {
21     const url = cursor
22       ? `/api/posts?cursor=${cursor}`
23       : "/api/posts";
24
25     const response = await fetch(url);
26     return response.json();
27 };
28
29 (async () => {
30     let count = 0;
```

```
31   for await (const post of infiniteScroll(fetchPosts)) {
32     console.log(post.title);
33     count++;
34
35     // 加载 100 条后停止
36     if (count >= 100) {
37       break;
38     }
39   }
40 }());
```

异步迭代器协议

手动实现异步迭代器。

```
1  class AsyncRange { ⚡ JavaScript
2    constructor(start, end) {
3      this.start = start;
4      this.end = end;
5    }
6
7    [Symbol.asyncIterator]() {
8      let current = this.start;
9
10     return {
11       async next() {
12         // 模拟异步操作
13         await new Promise(resolve => setTimeout(resolve, 100));
14
15         if (current < this.end) {
16           return { value: current++, done: false };
17         } else {
18           return { done: true };
19         }
20       }
21     };
22   }
23 }
24
25 // 使用
26 (async () => {
27   for await (const num of new AsyncRange(0, 5)) {
28     console.log(num); // 0, 1, 2, 3, 4 (每个间隔 100ms)
29   }
30 })();
```

组合异步迭代器

```
1  // 过滤 ⚡ JavaScript
2  async function* filter(asyncIterable, predicate) {
3    for await (const item of asyncIterable) {
4      if (predicate(item)) {
5        yield item;
```

```
6     }
7   }
8 }
9
10 // 映射
11 async function* map(asyncIterable, transform) {
12   for await (const item of asyncIterable) {
13     yield transform(item);
14   }
15 }
16
17 // 限制数量
18 async function* take(asyncIterable, count) {
19   let taken = 0;
20   for await (const item of asyncIterable) {
21     if (taken >= count) break;
22     yield item;
23     taken++;
24   }
25 }
26
27 // 组合使用
28 (async () => {
29   const numbers = new AsyncRange(0, 100);
30
31   const result = await Array.fromAsync(
32     take(
33       filter(
34         map(numbers, n => n * 2),
35         n => n % 3 === 0
36       ),
37       5
38     )
39   );
40
41   console.log(result); // [0, 6, 12, 18, 24]
42 })();
```

NOTE

`Array.fromAsync()` 是 ES2023 新增的方法，用于将异步迭代器转换为数组。

应用场景

场景	说明	示例
分页数据	逐页加载	无限滚动、懒加载
数据流处理	流式处理大数据	文件读取、网络流
实时数据	WebSocket 消息流	股票行情、聊天消息
数据库查询	游标遍历	大量数据查询
管道处理	数据转换管道	ETL、数据处理

本节介绍了高级异步模式，包括回调地狱的解决方案、发布-订阅模式、生产者-消费者模式以及异步迭代器与生成器。这些模式能够帮助你解决复杂的异步编程场景，写出更优雅、更可维护的代码。

6 定时器与异步调度

6.1 setTimeout / setInterval

6.2 requestAnimationFrame

6.3 setImmediate (Node.js)

6.4 process.nextTick (Node.js)

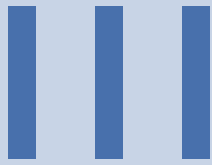
7 实战示例

7.1 API 请求封装

7.2 并发控制

7.3 重试机制

7.4 超时控制



TypeScript 类型系统与高级特性

IV

标准库与常用 API



浏览器端 JavaScript

VI

Node.js 与 Bun

8. Node.js 基础与演进 ·····	50
9. Bun 现代运行时 ·····	60
10. 文件系统与路径 ·····	72
11. HTTP 服务器 ·····	73
12. Express/Koa 框架 ·····	74
13. Node.js/Bun 高级主题 ·····	75

Chapter 8

Node.js 基础与演进

Node.js 是一个基于 Chrome V8 引擎的 JavaScript 运行时，让 JavaScript 可以运行在服务器端。它采用事件驱动、非阻塞 I/O 模型，使其轻量且高效。

1 Node.js 架构

1.1 单线程模型

```
1 // Node.js 是单线程的，但通过事件循环实现高并发
2 const http = require('http')
3
4 const server = http.createServer((req, res) => {
5   // 每个请求都在同一个线程中处理
6   console.log(`Request: ${req.url}`)
7   res.end('Hello World')
8 })
9
10 server.listen(3000)
11 console.log('Server running on port 3000')
```

为什么单线程？

优势	说明
无锁竞争	不需要处理多线程同步问题
内存占用低	无需为每个线程分配栈空间
上下文切换少	减少 CPU 开销
编程简单	避免死锁、竞态条件

CAUTION

单线程意味着 CPU 密集型任务会阻塞整个应用，需要使用 Worker Threads 或子进程。

1.2 非阻塞 I/O

```
1 const fs = require('fs')
2
3 // ❌ 阻塞 I/O (不推荐)
4 const data = fs.readFileSync('/path/to/file', 'utf8')
5 console.log(data) // 等待文件读取完成
6
```

```
7 //  非阻塞 I/O (推荐)
8 fs.readFile('/path/to/file', 'utf8', (err, data) => {
9   if (err) throw err
10  console.log(data) // 文件读取完成后回调
11 })
12
13 console.log('This runs immediately') // 不会等待文件读取
```

阻塞 vs 非阻塞：

```
1 阻塞 I/O: text
2 Thread: [Read File]————[Process]————[Response]
3         ↑ 等待 I/O 完成
4
5 非阻塞 I/O:
6 Thread: [Start Read]—[Other Work]—[Callback]—[Response]
7         ↑         ↑         ↑ 异步回调
8         发起请求   继续执行   I/O 完成
```

1.3 libuv 与事件循环

```
1 Node.js 架构: text
2
3 | JavaScript Code | ← V8 引擎执行 JS
4 |
5 | Node.js APIs   | ← fs, net, http 等
6 |
7 | libuv          | ← 事件循环、线程池
8 |
9 | C++ Bindings   | ← 系统调用封装
10 |
11 | Operating System | ← 文件系统、网络等
12 |
```

libuv 核心功能：

1. 事件循环：管理异步操作的生命周期
2. 线程池：处理文件系统、DNS 等阻塞操作（默认 4 个线程）
3. 异步 I/O：封装不同平台的 I/O API
4. 定时器：setTimeout、setInterval 的实现

NOTE

libuv 是 Node.js 跨平台的核心，提供了统一的事件循环和异步 I/O 抽象。

1.4 事件驱动

```
1 const EventEmitter = require('events')
2
3 class MyEmitter extends EventEmitter {}
4
5 const myEmitter = new MyEmitter()
```

 JavaScript

```
6
7 // 注册事件监听器
8 myEmitter.on('event', () => {
9   console.log('事件触发了! ')
10 })
11
12 // 触发事件
13 myEmitter.emit('event')
14
15 // 带参数的事件
16 myEmitter.on('greet', (name, age) => {
17   console.log(`Hello ${name}, you are ${age} years old`)
18 })
19
20 myEmitter.emit('greet', 'Alice', 25)
```

EventEmitter 常用方法:

方法	说明	示例
on/listen	注册监听器	emitter.on('data', handler)
emit	触发事件	emitter.emit('data', value)
once	只监听一次	emitter.once('connect', handler)
off/removeListener	移除监听器	emitter.off('data', handler)
removeAllListeners	移除所有	emitter.removeAllListeners()

2 版本演进

2.1 LTS 策略

Node.js 采用长期支持 (LTS) 策略, 确保生产环境的稳定性。

版本类型:

类型	支持周期	适用场景
Current	6 个月	测试新特性
Active LTS	12 个月	生产环境推荐
Maintenance LTS	18 个月	遗留系统维护
End-of-Life	无	不再支持

发布周期:

- 1 每年4月发布偶数版本 (v18, v20, v22...)
- 2 v18 → Current (6个月) → Active LTS (12个月) → Maintenance (18个月) → EOL

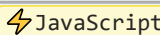
TIP

生产环境始终使用 Active LTS 或 Maintenance LTS 版本。

2.2 重要版本特性

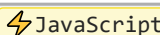
v14 (2020):

```
1 // 可选链操作符
2 const name = user?.profile?.name
3
4 // 空值合并运算符
5 const value = input ?? 'default'
6
7 // Promise.allSettled
8 Promise.allSettled([p1, p2, p3])
9
10 // 全局 this
11 globalThis === window // 浏览器
12 globalThis === global // Node.js
```



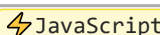
v16 (2021):

```
1 // fetch API (实验性)
2 const response = await fetch('https://api.example.com')
3
4 // AbortController
5 const controller = new AbortController()
6 setTimeout(() => controller.abort(), 5000)
7
8 fetch(url, { signal: controller.signal })
9
10 // Error.cause
11 throw new Error('Failed', { cause: originalError })
```



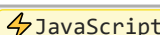
v18 (2022):

```
1 // fetch API (稳定)
2 const response = await fetch('https://api.example.com')
3 const data = await response.json()
4
5 // Test Runner (实验性)
6 import { test } from 'node:test'
7 import assert from 'node:assert'
8
9 test('adds 1 + 2 to equal 3', () => {
10   assert.strictEqual(1 + 2, 3)
11 })
12
13 // Web Crypto API
14 const key = await crypto.subtle.generateKey(...)
```



v20 (2023):

```
1 // Test Runner (稳定)
2 import { test, describe } from 'node:test'
3
4 describe('Math', () => {
```



```
5   test('addition', () => {
6     assert.strictEqual(1 + 2, 3)
7   })
8 })
9
10 // Permission Model (实验性)
11 // node --allow-read=src app.js
12
13 // Single Executable Applications
14 // pkg 工具的官方替代
```

v22 (2024):

⚡ JavaScript

```
1 // require() 支持 ES Modules
2 import { createRequire } from 'module'
3 const require = createRequire(import.meta.url)
4
5 // WebSocket (稳定)
6 const ws = new WebSocket('ws://localhost:8080')
7
8 // SQLite (内置)
9 import db from 'node:sqlite'
10 const database = db.openSync('./app.db')
```

NOTE

每个主要版本都带来重要的新特性和性能改进，建议定期升级。

2.3 版本对比

特性	v14	v16	v18	v20	v22
fetch API	✗	🔬	✅	✅	✅
Test Runner	✗	✗	🔬	✅	✅
Web Crypto	✗	✗	✅	✅	✅
Permission Model	✗	✗	✗	🔬	🔬
WebSocket	✗	✗	✗	✗	✅
SQLite	✗	✗	✗	✗	✅

图例：✗ 不支持 | 🔬 实验性 | ✅ 稳定支持

3 模块系统

3.1 CommonJS

⚡ JavaScript

```
1 // math.js - 导出
2 function add(a, b) {
```

```
3   return a + b
4 }
5
6 function subtract(a, b) {
7   return a - b
8 }
9
10 module.exports = { add, subtract }
11 // 或者
12 exports.add = add
13 exports.subtract = subtract
```

```
1 // app.js - 导入
2 const { add, subtract } = require('./math')
3
4 console.log(add(1, 2)) // 3
5 console.log(subtract(5, 3)) // 2
```

 JavaScript

特点:

- 同步加载: `require()` 是同步的
- 运行时解析: 在代码执行时解析模块
- 动态导入: 可以根据条件加载模块
- 缓存机制: 模块只加载一次, 后续从缓存获取

NOTE

CommonJS 是 Node.js 的原生模块系统, 至今仍广泛使用。

3.2 ES Modules

```
1 // math.mjs - 导出
2 export function add(a, b) {
3   return a + b
4 }
5
6 export function subtract(a, b) {
7   return a - b
8 }
9
10 // 默认导出
11 export default function multiply(a, b) {
12   return a * b
13 }
```

 JavaScript

```
1 // app.mjs - 导入
2 import { add, subtract } from './math.mjs'
3 import multiply from './math.mjs'
4
5 console.log(add(1, 2)) // 3
6 console.log(multiply(2, 3)) // 6
```

 JavaScript

在 `package.json` 中启用:

```
1 {  
2   "type": "module" // 启用 ES Modules  
3 }
```

特点:

- 异步加载: import 是异步的
- 编译时解析: 在代码执行前解析依赖
- 静态分析: 支持 Tree-Shaking
- 浏览器兼容: 与浏览器 ES Modules 一致

TIP

新项目推荐使用 ES Modules, 旧项目可以逐步迁移。

3.3 混合使用

```
1 // 在 ES Module 中使用 CommonJS  
2 import { createRequire } from 'module'  
3 const require = createRequire(import.meta.url)  
4  
5 const lodash = require('lodash')  
6  
7 // 在 CommonJS 中使用 ES Module (需要动态导入)  
8 async function loadModule() {  
9   const module = await import('./esm-module.mjs')  
10  return module.default  
11 }
```

兼容性对比:

特性	CommonJS	ES Modules
语法	require/module.exports	import/export
加载方式	同步	异步
解析时机	运行时	编译时
Tree-Shaking	✗	✓
动态导入	✓	✓ (import())
浏览器支持	✗	✓
文件扩展名	.js	.mjs 或 type: module

3.4 内置模块

```
1 // 核心模块 (无需安装)  
2 const fs = require('fs') // 文件系统  
3 const path = require('path') // 路径处理  
4 const http = require('http') // HTTP 服务器  
5 const os = require('os') // 操作系统信息  
6 const crypto = require('crypto') // 加密  
7 const events = require('events') // 事件发射器
```



```
8  const stream = require('stream') // 流处理
9  const util = require('util')     // 工具函数
10
11 // ES Modules 导入
12 import fs from 'fs'
13 import path from 'path'
14 import http from 'http'
```

常用内置模块：

模块	用途	常用方法
fs	文件操作	readFile, writeFile, mkdir
path	路径处理	join, resolve, basename
http	HTTP 服务	createServer, request
os	系统信息	platform, cpus, freemem
crypto	加密	createHash, randomBytes
events	事件	EventEmitter
stream	流	Readable, Writable
child_process	子进程	exec, spawn

4 全局对象

4.1 global

```
1  // global 是 Node.js 的全局对象（类似浏览器的 window）
2
3  // 设置全局变量
4  global.myVar = 'Hello'
5  console.log(global.myVar) // 'Hello'
6
7  // 全局函数
8  global.sayHi = () => console.log('Hi')
9  global.sayHi() // 'Hi'
10
11 // 注意：避免污染全局命名空间
```

⚡ JavaScript

CAUTION

尽量避免使用 global，会导致命名冲突和难以维护的代码。

4.2 process

```
1  // 进程信息
2  console.log(process.pid) // 进程 ID
3  console.log(process.version) // Node.js 版本
```

⚡ JavaScript

```

4 console.log(process.platform) // 操作系统平台
5 console.log(process.arch) // CPU 架构
6
7 // 环境变量
8 console.log(process.env.NODE_ENV) // 'development' or 'production'
9 console.log(process.env.PORT) // 端口号
10
11 // 命令行参数
12 console.log(process.argv) // ['node', 'script.js', 'arg1', 'arg2']
13
14 // 退出进程
15 process.exit(0) // 成功退出
16 process.exit(1) // 失败退出
17
18 // 事件监听
19 process.on('exit', (code) => {
20   console.log(`About to exit with code: ${code}`)
21 })
22
23 process.on('uncaughtException', (err) => {
24   console.error('Uncaught Exception:', err)
25   process.exit(1)
26 })

```

常用属性:

属性	说明	示例
argv	命令行参数	['node', 'app.js', '--port=3000']
env	环境变量	process.env.NODE_ENV
pid	进程 ID	12345
platform	平台	'linux', 'darwin', 'win32'
version	Node 版本	'v20.10.0'
cwd()	当前目录	process.cwd()

4.3 `__dirname` 和 `__filename`

```

1 // __dirname: 当前模块的目录名
2 console.log(__dirname) // '/home/user/project/src'
3
4 // __filename: 当前模块的文件名 (绝对路径)
5 console.log(__filename) // '/home/user/project/src/app.js'
6
7 // 常用组合
8 const path = require('path')
9 const configPath = path.join(__dirname, 'config.json')

```

JavaScript

ES Modules 中的等价物:

```
1 import { fileURLToPath } from 'url'
```

JavaScript

```
2 import { dirname } from 'path'
3
4 const __filename = fileURLToPath(import.meta.url)
5 const __dirname = dirname(__filename)
```

NOTE

`__dirname` 和 `__filename` 在 ES Modules 中不可用，需要使用 `import.meta.url`。

4.4 import.meta

```
1 // ES Modules 中的元信息
2
3 // 当前模块的 URL
4 console.log(import.meta.url) // 'file:///home/user/project/src/app.js'
5
6 // 解析路径
7 import { fileURLToPath } from 'url'
8 const __filename = fileURLToPath(import.meta.url)
9
10 // 判断是否为主模块
11 if (import.meta.url === `file://${process.argv[1]}`) {
12   console.log('这是主模块')
13 }
14
15 // 加载 JSON 文件
16 import { readFileSync } from 'fs'
17 const packageJson = JSON.parse(
18   readFileSync(new URL('../package.json', import.meta.url), 'utf8')
19 )
```

 JavaScript

import.meta 的优势：

- 标准化：ES Modules 标准的一部分
- 浏览器兼容：在浏览器中也可用
- 更灵活：可以传递任意元信息
- 类型安全：TypeScript 完整支持

本章完

Chapter 9

Bun 现代运行时

Bun 是一个现代化的 JavaScript 运行时，旨在成为 Node.js 的快速、一体化替代品。它由 Zig 编写，使用 JavaScriptCore 引擎，提供了卓越的性能和内置的工具链。

1 Bun 简介

1.1 为什么需要 Bun

Node.js 已经存在十多年，虽然成熟稳定，但也积累了一些问题：

Node.js 的痛点:

问题	说明
启动速度慢	大型项目启动需要数秒
包管理慢	npm install 可能需要几分钟
工具分散	需要 webpack、jest、eslint 等多个工具
性能瓶颈	V8 引擎在某些场景下不够快
内存占用高	大型应用内存消耗大

Bun 的解决方案:

1. 极速启动：比 Node.js 快 4-5 倍
2. 内置工具链：bundler、test runner、package manager
3. 高性能：基于 JavaScriptCore，某些场景快 2-3 倍
4. 原生 TypeScript：无需配置即可运行 .ts 文件
5. Web API 兼容：fetch、WebSocket、ReadableStream 等

TIP

Bun 的目标不是完全取代 Node.js，而是提供一个更快、更现代的替代选择。

1.2 性能对比

```
1 # 启动速度对比
2 $ time node app.js      # ~100ms
3 $ time bun app.js       # ~20ms (快 5 倍)
4
5 # 包安装速度
6 $ time npm install      # ~30s
7 $ time bun install      # ~2s (快 15 倍)
```

```
8
9 # HTTP 服务器吞吐量
10 $ wrk -t12 -c400 -d30s http://localhost:3000
11 # Node.js: ~50,000 req/sec
12 # Bun: ~150,000 req/sec (快 3 倍)
```

基准测试:

场景	Node.js	Bun	提升
启动时间	100ms	20ms	5x
npm install	30s	2s	15x
HTTP 请求	50k req/s	150k req/s	3x
TypeScript 编译	需 tsc	原生支持	∞
内存占用	100MB	60MB	40% ↓

NOTE

性能数据仅供参考，实际表现取决于具体应用场景。

1.3 设计理念

Bun 的核心原则:

- 1. 速度优先: 所有操作都追求极致性能
- 2. 开箱即用: 内置常用工具，减少配置
- 3. Web 标准: 优先实现 Web API，提高兼容性
- 4. 开发者体验: 简化工作流程，减少摩擦
- 5. 兼容性: 尽可能兼容 Node.js API

技术栈:



CAUTION

Bun 使用 JavaScriptCore 而非 V8，这带来了性能优势，但也可能导致某些 V8 特有行为不兼容。

2 Bun vs Node.js

2.1 架构差异

引擎对比：

特性	Node.js (V8)	Bun (JSC)
引擎	Google V8	Apple JavaScriptCore
语言	C++	Zig
JIT 编译	TurboFan	DFG + FTL
垃圾回收	Orinoco	Mark-and-Sweep
启动速度	中等	极快
峰值性能	极高	高
内存占用	较高	较低

为什么 JSC 更快？

1. 更简单的 JIT：JSC 的编译器层级更少，启动更快
2. 更低的内存开销：JSC 的对象模型更紧凑
3. 更好的冷启动：JSC 在首次执行时优化更快
4. Zig 的优势：系统级语言，无运行时开销

NOTE

V8 在长时间运行的应用中可能表现更好，但 JSC 在启动速度和内存效率上占优。

2.2 API 兼容性

完全兼容：

```
1 // 这些 API 在 Bun 中完全兼容 Node.js
2 const fs = require('fs')
3 const path = require('path')
4 const http = require('http')
5 const crypto = require('crypto')
6
7 // ES Modules
8 import fs from 'fs'
9 import path from 'path'
```

部分兼容：

```
1 // child_process - 部分 API 未实现
2 const { exec } = require('child_process') // ✅
3 const { fork } = require('child_process') // ⚠️ 实验性
4
5 // cluster - 不支持
6 const cluster = require('cluster') // ❌ 未实现
7
```

```
8 // worker_threads - 实验性支持
9 const { Worker } = require('worker_threads') // ⚠ 实验性
```

Bun 特有 API:

```
1 // Bun.serve - 高性能 HTTP 服务器 ⚡ JavaScript
2 const server = Bun.serve({
3   port: 3000,
4   fetch(request) {
5     return new Response('Hello World')
6   },
7 })
8
9 // Bun.file - 惰性文件读取
10 const file = Bun.file('package.json')
11 console.log(file.size) // 不读取文件内容
12 const text = await file.text() // 惰性读取
13
14 // Bun.spawn - 高性能子进程
15 const proc = Bun.spawn(['ls', '-la'])
16 const stdout = await new Response(proc.stdout).text()
17
18 // Bun.hash - 快速哈希
19 const hash = Bun.hash('hello') // Wyhash 算法
```

兼容性对比表:

模块	Node.js	Bun	备注
fs	✓	✓	完全兼容
path	✓	✓	完全兼容
http	✓	✓	完全兼容
https	✓	✓	完全兼容
crypto	✓	✓	完全兼容
child_process	✓	⚠	部分支持
cluster	✓	✗	未实现
worker_threads	✓	⚠	实验性
Bun.*	✗	✓	Bun 特有

图例: ✓ 完全支持 | ⚠ 部分支持 | ✗ 不支持

2.3 性能优势

Bun 的优势场景:

1. CLI 工具: 快速启动, 即时响应
2. 脚本执行: 快速运行一次性任务
3. 开发服务器: 热更新更快, 重启更快
4. TypeScript 项目: 无需编译, 直接运行

5. 高并发 I/O: 更高的吞吐量

Node.js 的优势场景:

1. CPU 密集型: V8 的优化编译器更强
2. 长期运行服务: 成熟的生态和稳定性
3. 复杂应用: 更多的库和工具支持
4. 企业级应用: LTS 支持和商业保障

TIP

选择 Bun 还是 Node.js 取决于具体需求: 追求速度和开发体验选 Bun, 追求稳定性和生态选 Node.js。

3 Bun 特有 API

3.1 Bun.serve

Bun 内置的高性能 HTTP 服务器, 比 Express/Koa 快得多。

```
1 // 基本用法
2 const server = Bun.serve({
3   port: 3000,
4   fetch(request) {
5     return new Response('Hello World')
6   },
7 })
8
9 console.log(`Listening on http://localhost:${server.port}`)
```

路由处理:

```
1 Bun.serve({
2   port: 3000,
3   fetch(request) {
4     const url = new URL(request.url)
5
6     if (url.pathname === '/') {
7       return new Response('Home Page')
8     }
9
10    if (url.pathname === '/api/users') {
11      return Response.json([
12        { id: 1, name: 'Alice' },
13        { id: 2, name: 'Bob' },
14      ])
15    }
16
17    return new Response('Not Found', { status: 404 })
18  },
19 })
```

静态文件服务:


```
1 Bun.serve({⚡ JavaScript
2   port: 3000,
3   async fetch(request) {
4     const url = new URL(request.url)
5     const filePath = `./public${url.pathname}`
6
7     try {
8       const file = Bun.file(filePath)
9       if (await file.exists()) {
10        return new Response(file)
11      }
12    } catch {}
13
14    return new Response('Not Found', { status: 404 })
15  },
16 })
```

WebSocket 支持:

```
1 Bun.serve({⚡ JavaScript
2   port: 3000,
3   fetch(request, server) {
4     if (server.upgrade(request)) {
5       return // WebSocket 连接已升级
6     }
7     return new Response('Regular HTTP')
8   },
9   websocket: {
10    open(ws) {
11      console.log('Client connected')
12      ws.send('Welcome!')
13    },
14    message(ws, message) {
15      console.log('Received:', message)
16      ws.send(`Echo: ${message}`)
17    },
18    close(ws, code, reason) {
19      console.log('Client disconnected')
20    },
21  },
22 })
```

NOTE

Bun.serve 基于底层 C API，性能远超基于 Node.js http 模块的框架。

3.2 Bun.file

惰性文件读取 API，只在需要时才读取文件内容。

```
1 // 创建文件引用（不读取内容）⚡ JavaScript
2 const file = Bun.file('package.json')
3
```

```
4 // 获取元信息（不读取内容）
5 console.log(file.size) // 文件大小
6 console.log(file.type) // MIME 类型
7 console.log(await file.exists()) // 是否存在
8
9 // 读取内容（惰性加载）
10 const text = await file.text() // 文本
11 const json = await file.json() // JSON
12 const arrayBuffer = await file.arrayBuffer() // ArrayBuffer
13 const blob = await file.blob() // Blob
```

性能优势：

```
1 // ❌ Node.js - 立即读取整个文件
2 const data = fs.readFileSync('large-file.txt', 'utf8')
3
4 // ✅ Bun - 惰性读取，按需加载
5 const file = Bun.file('large-file.txt')
6 const size = file.size // 不读取内容
7 const firstLine = await file.text().then(t => t.split('\n')[0])
```

流式处理：

```
1 // 大文件流式传输
2 Bun.serve({
3   port: 3000,
4   fetch() {
5     const file = Bun.file('video.mp4')
6     return new Response(file.stream(), {
7       headers: {
8         'Content-Type': 'video/mp4',
9         'Content-Length': file.size,
10      },
11    })
12  },
13 })
```

3.3 Bun.spawn

高性能子进程 API，比 `child_process` 更快。

```
1 // 基本用法
2 const proc = Bun.spawn(['ls', '-la'])
3
4 // 等待完成
5 await proc.exited
6 console.log('Exit code:', proc.exitCode)
7
8 // 读取输出
9 const stdout = await new Response(proc.stdout).text()
10 const stderr = await new Response(proc.stderr).text()
11
12 console.log('Output:', stdout)
```

传入输入:

```
1 const proc = Bun.spawn(['cat'], {  
2   stdin: 'Hello World\n',  
3   stdout: 'pipe',  
4   stderr: 'pipe',  
5 })  
6  
7 const output = await new Response(proc.stdout).text()  
8 console.log(output) // 'Hello World\n'
```

 JavaScript

环境变量:

```
1 const proc = Bun.spawn(['node', '-v'], {  
2   env: {  
3     ...process.env,  
4     NODE_ENV: 'production',  
5   },  
6   cwd: '/path/to/directory',  
7 })  
8  
9 await proc.exited
```

 JavaScript

性能对比:

```
1 // ❌ Node.js - 较慢  
2 const { exec } = require('child_process')  
3 exec('ls -la', (error, stdout) => {  
4   console.log(stdout)  
5 })  
6  
7 // ✅ Bun - 更快  
8 const proc = Bun.spawn(['ls', '-la'])  
9 const stdout = await new Response(proc.stdout).text()  
10 console.log(stdout)
```

 JavaScript

TIP

Bun.spawn 避免了 Node.js child_process 的序列化开销，性能提升显著。

3.4 其他 Bun API

Bun.hash:

```
1 // 快速哈希函数 (Wyhash 算法)  
2 const hash = Bun.hash('hello world')  
3 console.log(hash) // BigInt  
4  
5 // 指定种子  
6 const hash2 = Bun.hash('hello', 12345)
```

 JavaScript

Bun.write:

```
1 // 高效写入文件  
2 await Bun.write('output.txt', 'Hello World')
```

 JavaScript

```

3  await Bun.write('data.json', JSON.stringify({ key: 'value' }))
4
5  // 写入 Buffer
6  const buffer = new Uint8Array([1, 2, 3])
7  await Bun.write('binary.bin', buffer)

```

Bun.sleep:

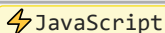
```

1  // 更简洁的 setTimeout 
2  await Bun.sleep(1000) // 睡眠 1 秒
3
4  // 等价于
5  await new Promise(resolve => setTimeout(resolve, 1000))

```

Bun.which:

```

1  // 查找可执行文件路径 
2  const nodePath = Bun.which('node')
3  console.log(nodePath) // '/usr/bin/node'
4
5  const pythonPath = Bun.which('python3')

```

完整 API 列表:

API	用途	示例
Bun.serve	HTTP 服务器	Bun.serve({ fetch })
Bun.file	惰性文件	Bun.file('path')
Bun.spawn	子进程	Bun.spawn(['cmd'])
Bun.hash	哈希	Bun.hash('data')
Bun.write	写入文件	Bun.write(path, data)
Bun.sleep	延迟	Bun.sleep(ms)
Bun.which	查找命令	Bun.which('node')
Bun.env	环境变量	Bun.env.NODE_ENV

4 迁移指南

4.1 Node.js → Bun

步骤 1：安装 Bun

```

1  # macOS/Linux 
2  curl -fsSL https://bun.sh/install | bash
3
4  # Windows
5  powershell -c "irm bun.sh/install.ps1|iex"
6
7  # 验证安装

```

```
8 bun --version
```

步骤 2：替换运行命令

```
1 # Node.js
2 node app.js
3 node --watch app.js
4
5 # Bun
6 bun app.js
7 bun --watch app.js
```

步骤 3：替换包管理器

```
1 # npm
2 npm install
3 npm run dev
4
5 # Bun
6 bun install
7 bun run dev
```

步骤 4：检查兼容性

```
1 # 运行项目
2 bun app.js
3
4 # 查看警告
5 # Bun 会提示不兼容的 API
```

NOTE

大多数 Node.js 项目可以直接用 Bun 运行，无需修改代码。

4.2 兼容性问题

已知不兼容：

问题	影响	解决方案
cluster 模块	多进程	使用 Bun.spawn 或外部工具
某些 native addons	C++ 扩展	寻找纯 JS 替代
vm 模块	沙箱执行	使用 isolated-vm
inspector	调试协议	使用 Chrome DevTools
perf_hooks	性能监控	使用 Bun 内置工具

常见陷阱：

```
1 // ❌ 依赖 V8 特定行为
2 const v8 = require('v8') // Bun 不支持
3
4 // ✅ 使用标准 API
5 const crypto = require('crypto') // 两者都支持
```

```
6
7 // ❌ 使用 cluster
8 const cluster = require('cluster') // Bun 不支持
9
10 // ✅ 使用多个 Bun 实例
11 // 在 shell 中启动多个实例
```

CAUTION

在生产环境使用 Bun 前，务必进行充分的测试。

4.3 最佳实践

渐进式采用：

```
1 # 1. 先在开发环境试用
2 bun run dev
3
4 # 2. 运行测试
5 bun test
6
7 # 3. 构建生产版本
8 bun build ./src/index.ts --outdir ./dist
9
10 # 4. 生产环境仍使用 Node.js (初期)
11 node dist/index.js
```

 Bash

利用 Bun 特性：

```
1 // 直接使用 TypeScript, 无需编译
2 // index.ts
3 import { serve } from 'bun'
4
5 serve({
6   port: 3000,
7   fetch: () => new Response('Hello'),
8 })
9
10 // 运行
11 bun index.ts // 无需 tsc
```

 TypeScript

使用 Bun 内置工具：

```
1 # Bundler
2 bun build ./src/index.ts --outdir ./dist
3
4 # Test Runner
5 bun test
6
7 # Package Manager
8 bun install
9 bun add lodash
10 bun remove lodash
11
```

 Bash

```
12 # Hot Reload
13 bun --watch app.ts
```

性能优化:

```
1 // 使用 Bun.file 惰性读取
2 const config = Bun.file('config.json')
3
4 // 使用 Bun.serve 替代 Express
5 Bun.serve({ fetch })
6
7 // 使用 Bun.spawn 替代 child_process
8 const proc = Bun.spawn(['cmd'])
```

 JavaScript

TIP

充分利用 Bun 的特有 API，可以获得最佳性能和开发体验。

本章完

Chapter 10

文件系统与路径

Chapter 11

HTTP 服务器

Chapter 12

Express/Koa 框架

Chapter 13

Node.js/Bun 高级主题

VII

JavaScript 工程化

14. 包管理器	77
15. 构建工具	94
16. 代码质量	95
17. 测试框架	96
18. 部署与 CI/CD	97

Chapter 14

包管理器

JavaScript 生态系统依赖强大的包管理器来管理依赖、脚本和工具链。从 npm 到 Bun，包管理器经历了多次演进。

1 npm

npm (Node Package Manager) 是 Node.js 的默认包管理器，也是世界上最大的软件注册表。

1.1 安装策略与 node_modules

传统安装方式：

```
1 # npm 直接安装到 node_modules
2 npm install express
3
4 # 项目结构
5 project/
6 |— node_modules/      # 所有依赖都在这里
7 |   |— express/
8 |   |— lodash/
9 |   |— ... (数百个文件夹)
10 |— package.json
11 |— package-lock.json
```

问题：

问题	说明	影响
磁盘占用大	每个项目都有完整副本	100 个项目 = 5GB+
安装慢	需要复制大量文件	大型项目需要分钟级
嵌套深	依赖树嵌套导致路径过长	Windows 路径长度限制
Phantom deps	可以访问未声明的依赖	隐藏 bug

CAUTION

npm 的传统安装方式是 JavaScript 生态“node_modules 黑洞”的根源。

1.2 命令简写

npm 提供了丰富的命令简写，提高开发效率。

1	# 安装依赖	
2	<code>npm install express</code>	# 完整写法
3	<code>npm i express</code>	# 简写
4	<code>npm add express</code>	# 另一种简写（不常用）
5		
6	# 开发依赖	
7	<code>npm install --save-dev typescript</code>	# 完整
8	<code>npm i -D typescript</code>	# 简写
9	<code>npm i --dev typescript</code>	# 另一种
10		
11	# 全局安装	
12	<code>npm install -g nodemon</code>	# 完整
13	<code>npm i -g nodemon</code>	# 简写
14		
15	# 卸载	
16	<code>npm uninstall express</code>	# 完整
17	<code>npm rm express</code>	# 简写
18	<code>npm remove express</code>	# 另一种
19		
20	# 运行脚本	
21	<code>npm run dev</code>	# 完整
22	<code>npm run build</code>	# 完整
23		
24	# 查看信息	
25	<code>npm list</code>	# 列出依赖
26	<code>npm ls</code>	# 简写
27	<code>npm outdated</code>	# 查看过时依赖
28	<code>npm update</code>	# 更新依赖
29	<code>npm up</code>	# 简写

常用简写对照表：

完整命令	简写	说明
<code>install</code>	<code>i</code>	安装依赖
<code>uninstall</code>	<code>rm/remove/un</code>	卸载依赖
<code>--save-dev</code>	<code>-D</code>	开发依赖
<code>--global</code>	<code>-g</code>	全局安装
<code>list</code>	<code>ls</code>	列出依赖
<code>update</code>	<code>up</code>	更新依赖
<code>init</code>	<code>create</code>	初始化项目
<code>test</code>	<code>t/tst</code>	运行测试

TIP

团队项目中应统一使用完整命令或简写，避免混用造成混乱。

1.3 package.json

```
1  {
2    "name": "my-project",
3    "version": "1.0.0",
4    "description": "My awesome project",
5    "main": "index.js",
6    "scripts": {
7      "dev": "vite",
8      "build": "vite build",
9      "test": "vitest",
10     "lint": "eslint ."
11   },
12   "dependencies": {
13     "express": "^4.18.0",
14     "lodash": "~4.17.21"
15   },
16   "devDependencies": {
17     "typescript": "^5.0.0",
18     "vitest": "^1.0.0"
19   },
20   "engines": {
21     "node": ">=18.0.0"
22   }
23 }
```

关键字段：

字段	说明	示例
name	包名	my-package
version	版本号	1.0.0
dependencies	生产依赖	express: ^ 4.18.0
devDependencies	开发依赖	typescript: ^ 5.0.0
scripts	脚本命令	build: vite build
engines	引擎要求	node: >=18
peerDependencies	同伴依赖	react: ^ 18.0.0

NOTE

^ 表示兼容版本（允许小版本升级），~ 表示补丁版本（只允许补丁升级）。

1.4 scripts

```
1  {
2    "scripts": {
3      "dev": "vite",
4      "build": "vite build",
5      "preview": "vite preview",
6      "test": "vitest",
7      "test:coverage": "vitest --coverage",
8      "lint": "eslint src --ext .ts,.vue",
9    }
10 }
```

```
9   "lint:fix": "eslint src --ext .ts,.vue --fix",
10  "type-check": "tsc --noEmit",
11  "clean": "rm -rf dist",
12  "prebuild": "npm run clean",
13  "postbuild": "echo Build complete!"
14  }
15 }
```

生命周期钩子：

```
1 # pre/post 钩子自动执行
2 npm run build
3 # 执行顺序: prebuild → build → postbuild
4
5 # 常用组合
6 npm run lint && npm run test && npm run build
```

 Bash

传递参数：

```
1 # 使用 -- 传递参数
2 npm run test -- --reporter=verbose
3 npm run dev -- --port 3000
4
5 # 或使用环境变量
6 PORT=3000 npm run dev
```

 Bash

1.5 依赖管理

```
1 # 安装依赖
2 npm install express          # 生产依赖
3 npm install -D typescript    # 开发依赖 (--save-dev)
4 npm install lodash@4.17.21  # 指定版本
5
6 # 更新依赖
7 npm update                   # 更新所有依赖
8 npm update express           # 更新指定依赖
9 npm outdated                 # 查看可更新的依赖
10
11 # 卸载依赖
12 npm uninstall express
13 npm uninstall -D typescript
14
15 # 清理缓存
16 npm cache clean --force
```

 Bash

版本控制：

```
1 # 语义化版本
2 npm version patch            # 1.0.0 → 1.0.1
3 npm version minor            # 1.0.0 → 1.1.0
4 npm version major            # 1.0.0 → 2.0.0
5
6 # 查看版本
```

 Bash


```
7 npm view express versions
8 npm view express latest
```

TIP

始终使用锁文件（package-lock.json）确保依赖版本一致。

1.6 npx

npx（npm package executor）是 npm 5.2+ 内置的工具执行器，可以运行本地或远程的包。

基本用法：

```
1 # 运行本地安装的包
2 npx jest # 使用项目中的 jest
3 npx typescript # 使用项目中的 tsc
4
5 # 运行未安装的包（临时下载）
6 npx create-react-app my-app # 自动下载并执行
7 npx cowsay "Hello" # 有趣的命令行工具
8
9 # 指定版本
10 npx eslint@8.0.0 src/
11
12 # 忽略本地版本，始终使用最新
13 npx --ignore-existing create-react-app my-app
```

 Bash

工作原理：

```
1 npx 执行流程：
2 1. 检查 node_modules/.bin/ 中是否有该命令
3 2. 如果有，直接执行
4 3. 如果没有，从 npm registry 临时下载
5 4. 执行完成后删除（不保留在项目中）
```

 text

常见用途：

```
1 # 1. 初始化项目
2 npx create-next-app@latest
3 npx create-vite@latest
4 npx @angular/cli new my-app
5
6 # 2. 代码格式化
7 npx prettier --write src/
8 npx eslint src/ --fix
9
10 # 3. 测试
11 npx jest
12 npx mocha test/
13
14 # 4. 构建
15 npx webpack
16 npx vite build
17
```

 Bash

```

18 # 5. 数据库迁移
19 npx prisma migrate dev
20 npx typeorm migration:run
21
22 # 6. 临时工具
23 npx http-server ./dist # 快速启动静态服务器
24 npx serve ./dist # 另一种选择
25 npx ngrok http 3000 # 内网穿透

```

npx vs npm scripts:

```

1 // package.json
2 {
3   "scripts": {
4     // ✅ 推荐: 在 scripts 中定义
5     "lint": "eslint src/",
6     "test": "jest",
7     "build": "vite build",
8
9     // ❌ 不推荐: 每次都写 npx
10    "lint": "npx eslint src/",
11  }
12 }

```

```

1 # 使用 scripts
2 npm run lint # ✅ 简洁
3
4 # 直接使用 npx
5 npx eslint src/ # ⚠️ 每次都要写

```

优势:

优势	说明	示例
无需全局安装	避免污染全局命名空间	<code>npx create-react-app</code>
版本隔离	每个项目可以使用不同版本	project-a 用 <code>jest@27</code> , project-b 用 <code>jest@29</code>
临时执行	用完即删, 不占用空间	<code>npx cowsay</code>
最新版本	总是使用最新版	<code>npx create-next-app@latest</code>

NOTE

npx 是现代 JavaScript 开发的核心工具, 几乎每天都会用到。

1.7 Bun 的等价物

Bun 提供了类似的工具执行功能。

Bun x:

```

1 # Bun x 类似于 npx
2 bunx create-react-app my-app
3 bunx jest
4 bunx eslint src/

```

```

5
6 # 简写
7 bun x create-vite@latest

```

区别:

特性	npm	bunx
来源	npm 内置	Bun 内置
速度	中等	更快
缓存	npm 缓存	Bun 缓存
兼容性	所有 npm 包	所有 npm 包
额外功能	无	可以利用 Bun 性能优势

实际使用:

```

1 # 大多数情况下, npx 和 bunx 可以互换
2 npx create-next-app@latest # ✓
3 bunx create-next-app@latest # ✓ 更快
4
5 # 但如果使用 Bun 运行时, 建议用 bunx
6 bunx --bun create-next-app@latest # 使用 Bun 运行时

```

TIP

如果你已经安装了 Bun, 可以用 bunx 替代 npx 获得更快的速度。

npm workspaces 支持 monorepo 项目管理。

```

1 // package.json (root)
2 {
3   "name": "monorepo-root",
4   "private": true,
5   "workspaces": [
6     "packages/*"
7   ]
8 }

```

```

1 monorepo/
2 |— package.json
3 |— packages/
4 |   |— pkg-a/
5 |   |   |— package.json
6 |   |   |— src/
7 |   |— pkg-b/
8 |       |— package.json
9 |       |— src/

```

```

1 # 安装所有 workspace 的依赖
2 npm install
3
4 # 在特定 workspace 中运行命令
5 npm run build --workspace=pkg-a

```

```
6
7 # 添加跨 workspace 依赖
8 npm install pkg-a --workspace=pkg-b
```

优势：

- 共享依赖：多个项目共享相同的依赖
- 本地链接：workspace 之间自动链接
- 统一脚本：可以批量执行脚本
- 简化维护：统一管理多个相关项目

NOTE

npm workspaces 是 npm 7+ 的特性，之前需要使用 Lerna 或 Yarn workspaces。

2 yarn

Yarn 是由 Facebook 开发的替代 npm 的包管理器，提供了更快的安装速度和更好的依赖管理。

2.1 Yarn 的现状

历史回顾：

1	2016: Yarn v1 发布，解决 npm v3/v4 的性能问题	text
2	2017: npm v5 发布，缩小了与 Yarn 的差距	
3	2019: Yarn v2 (Berry) 发布，引入 PnP 等重大变更	
4	2020+: npm 持续改进，pnpm 崛起，Bun 出现	
5	2024: Yarn 市场份额下降，pnpm 和 Bun 更受欢迎	

当前市场地位：

包管理器	2020 年份额	2024 年份额	趋势
npm	~70%	~65%	稳定
yarn	~20%	~10%	下降 ↓
pnpm	~5%	~15%	上升 ↑
Bun	~0%	~8%	快速增长 ↑↑

为什么 Yarn 使用减少？

原因	说明
npm 改进	npm v5+ 性能大幅提升，锁文件、workspace 等特性跟进
pnpm 崛起	节省空间、严格模式、monorepo 支持更好
Bun 出现	极速安装、一体化体验、开发者喜爱
Yarn v2 变更	PnP 破坏兼容性，迁移成本高
生态惯性	大部分项目默认使用 npm

NOTE

Yarn 仍然是一个优秀的包管理器，但在新项目中选择率下降。现有 Yarn 项目可以继续使用，无需迁移。

2.2 Yarn Classic vs Yarn Berry

Yarn Classic (v1):

```
1 # 安装
2 npm install -g yarn
3
4 # 基本命令
5 yarn add express
6 yarn add -D typescript
7 yarn remove express
8 yarn install
```

Yarn Berry (v2/v3/v4):

```
1 # 安装
2 corepack enable
3 yarn set version stable
4
5 # 新的命令语法
6 yarn add express
7 yarn add -D typescript
8 yarn remove express
9 yarn install
```

主要区别:

特性	Yarn v1	Yarn v2+
锁文件	yarn.lock	yarn.lock
node_modules	传统文件夹	Plug'n'Play
缓存	全局缓存	零安装
配置	.yarnrc	.yarnrc.yml
工作区	支持	增强支持

CAUTION

Yarn Berry 引入了重大变更，迁移需要谨慎评估。

2.3 lock 文件

```
1 # yarn.lock 示例
2 express@^4.18.0:
3   version "4.18.2"
4   resolved "https://registry.yarnpkg.com/express/-/express-4.18.2.tgz#..."
5   integrity sha512-...
6 dependencies:
7   accepts "~1.3.8"
```

```
8 body-parser "1.20.1"
```

作用：

- 锁定版本：确保所有开发者使用相同的依赖版本
- 快速安装：直接从锁文件解析依赖树
- 一致性：避免 “works on my machine” 问题

TIP

始终将 lock 文件提交到版本控制系统。

2.4 workspaces

```
1 // package.json (root)
2 {
3   "private": true,
4   "workspaces": [
5     "packages/*"
6   ]
7 }
```

{ } JSON

```
1 # Yarn workspaces 命令
2 yarn workspace @myorg/pkg-a add express
3 yarn workspace @myorg/pkg-b run build
4
5 # 在所有 workspace 中运行
6 yarn workspaces foreach run build
```

Bash

优势：

- 提升依赖：自动 hoist 共享依赖到根目录
- 本地链接：workspace 之间自动链接
- 并行安装：更快的安装速度

2.5 Plug' n' Play (PnP)

Yarn Berry 的 PnP 模式消除了 node_modules 文件夹。

```
1 # .yarnrc.yml
2 nodeLinker: pnp
```

YAML

优势：

- 磁盘空间：减少重复依赖，节省空间
- 安装速度：无需复制文件到 node_modules
- 确定性：完全确定的依赖解析
- phantom dependencies：防止访问未声明的依赖

劣势：

- 兼容性：某些工具可能不支持 PnP
- 学习曲线：需要理解新的工作原理

NOTE

PnP 是 Yarn 的创新特性，但 adoption 仍然有限。

2.6 缓存

```
1 # Yarn 全局缓存
2 yarn cache dir      # 查看缓存目录
3 yarn cache clean    # 清理缓存
4 yarn cache list     # 列出缓存
5
6 # 离线模式
7 yarn install --offline # 使用缓存离线安装
```

⌨️ Bash

Zero-Installs:

Yarn Berry 支持将所有依赖提交到 Git，实现零安装。

```
1 # .yarnrc.yml
2 enableGlobalCache: false
```

⚙️ YAML

```
1 # 所有依赖存储在 .yarn/cache/
2 git add .yarn/cache/
3 git commit -m "Add dependencies"
4
5 # 克隆后无需安装
6 git clone repo
7 cd repo
8 yarn run build # 直接运行!
```

⌨️ Bash

TIP

Zero-Installs 适合小型项目，大型项目可能导致 Git 仓库过大。

3 pnpm

pnpm (performant npm) 是一个快速、节省磁盘空间的包管理器。

3.1 硬链接与符号链接

pnpm 使用内容寻址存储和硬链接来节省磁盘空间，类似于 Java 的 Maven 本地仓库。

Maven 对比:

```
1 Java Maven:
2 ~/.m2/repository/      # 全局仓库
3   org/springframework/
4     spring-core/5.3.0/
5     spring-web/5.3.0/
6
7 project-a/pom.xml      # 项目引用全局仓库
8 project-b/pom.xml      # 共享相同的依赖
```

text

```
9
10 JavaScript pnpm:
11 ~/.pnpm-store/v3/          # 全局仓库 (类似 .m2)
12   files/ab/cd/lodash-4.17.21/
13   files/ef/gh/express-4.18.2/
14
15 project-a/node_modules/    # 通过符号链接引用
16 project-b/node_modules/    # 共享相同的依赖
```

传统方式 (npm/yarn) :

```
1 project-a/node_modules/lodash/    # 完整副本
2 project-b/node_modules/lodash/    # 完整副本
3 project-c/node_modules/lodash/    # 完整副本
4 # 每个项目都有一份完整的 lodash 副本
5 # 100个项目 = 100份副本 = 浪费空间
```

[text](#)

pnpm 方式:

```
1 ~/.pnpm-store/v3/files/ab/cd/lodash-4.17.21/  # 只有一个副本
2
3 project-a/node_modules/.pnpm/lodash@4.17.21/ -> 符号链接到 store
4 project-b/node_modules/.pnpm/lodash@4.17.21/ -> 符号链接到 store
5 project-c/node_modules/.pnpm/lodash@4.17.21/ -> 符号链接到 store
6 # 只有一个副本, 通过符号链接共享
7 # 100个项目 = 1份副本 = 节省90%空间
```

[text](#)

工作流程:

```
1 # 1. 首次安装
2 pnpm install express
3 # ↓
4 # 下载 express 到 ~/.pnpm-store/
5 # 创建符号链接到 node_modules/
6
7 # 2. 第二个项目安装同样的依赖
8 pnpm install express
9 # ↓
10 # 检测到 ~/.pnpm-store/ 中已有 express
11 # 直接创建符号链接, 无需下载!
12
13 # 3. 离线安装
14 pnpm install --offline
15 # ↓
16 # 完全从本地 store 读取, 无需网络
```

[🖥️ Bash](#)

磁盘空间对比:

包管理器	100 个项目	节省空间	原理
npm	~5GB	0%	每个项目完整副本
yarn	~5GB	0%	每个项目完整副本
pnpm	~500MB	90%	全局 store + 符号链接

Bun	~500MB	90%	全局 cache + 硬链接
-----	--------	-----	----------------

NOTE

pnpm 和 Bun 的缓存机制类似于 Maven/Gradle，避免了重复下载和存储。

3.2 严格模式

pnpm 默认启用严格模式，防止 phantom dependencies。

Phantom Dependencies 问题：

```
1 // package.json
2 {
3   "dependencies": {
4     "express": "^4.18.0"
5   }
6 }
7
8 // index.js
9 const express = require('express') // ✓ 已声明
10 const lodash = require('lodash') // ✗ 未声明，但可能工作
11 // 因为 lodash 是 express 的依赖
```

 JavaScript

pnpm 的行为：

```
1 const lodash = require('lodash') // ✗ Error!
2 // Cannot find module 'lodash'
```

 JavaScript

解决方案：

```
1 # 显式安装
2 pnpm add lodash
3
4 # 或配置允许（不推荐）
5 # .npmrc
6 shamefully-hoist=true
```

 Bash**TIP**

严格模式强制显式声明所有依赖，提高项目的可维护性。

3.3 基本用法

```
1 # 安装
2 pnpm add express      # 生产依赖
3 pnpm add -D typescript # 开发依赖
4 pnpm add lodash@4.17.21 # 指定版本
5
6 # 安装所有依赖
7 pnpm install
8
9 # 更新
```

 Bash

```
10 pnpm update
11 pnpm update express
12
13 # 卸载
14 pnpm remove express
15
16 # 运行脚本
17 pnpm run dev
18 pnpm run build
```

workspace 支持:

```
1 # pnpm workspace
2 pnpm --filter @myorg/pkg-a add express
3 pnpm --filter @myorg/pkg-b run build
4
5 # 所有 workspace
6 pnpm -r run build
```

 Bash

3.4 性能优势

安装速度:

```
1 # 首次安装 (冷缓存)
2 npm install # ~30s
3 yarn install # ~25s
4 pnpm install # ~20s
5
6 # 二次安装 (有缓存)
7 npm install # ~10s
8 yarn install # ~8s
9 pnpm install # ~2s ← 显著更快
```

 Bash

原因:

1. 内容寻址存储: 避免重复下载
2. 硬链接: 无需复制文件
3. 并行安装: 充分利用 CPU
4. 智能缓存: 高效的缓存策略

NOTE

pnpm 在大型项目和 monorepo 中性能优势更明显。

4 Bun

Bun 内置了超快速的包管理器, 旨在替代 npm/yarn/pnpm。

4.1 极速安装

```
1 # Bun 包管理器命令
```

 Bash

2	<code>bun install</code>	# 安装依赖
3	<code>bun add express</code>	# 添加依赖
4	<code>bun add -d typescript</code>	# 开发依赖
5	<code>bun remove express</code>	# 移除依赖
6	<code>bun update</code>	# 更新依赖

性能对比：

1	# 安装大型项目 (1000+ 依赖)	 Bash
2	<code>npm install</code>	# ~30s
3	<code>yarn install</code>	# ~25s
4	<code>pnpm install</code>	# ~20s
5	<code>bun install</code>	# ~2s ← 快 10-15 倍

为什么这么快？

1. Zig 编写：系统级语言，无运行时开销
2. 并行下载：充分利用网络带宽
3. 高效缓存：智能缓存策略（类似 Maven）
4. 优化算法：更快的依赖解析
5. 硬链接：避免文件复制

Bun 的缓存机制：

1	Bun Cache:	
2	<code>~/.bun/install/cache/</code>	# 全局缓存（类似 ~/.m2 或 ~/.pnpm-store）
3	<code>express@4.18.2/</code>	
4	<code>lodash@4.17.21/</code>	
5	<code>react@18.2.0/</code>	
6		
7	<code>project-a/node_modules/</code>	# 通过硬链接引用缓存
8	<code>project-b/node_modules/</code>	# 共享相同的缓存
9		
10	# 首次安装：下载并缓存	
11	<code>bun install express</code>	
12	# ↓ 下载到 ~/.bun/install/cache/	
13	# ↓ 硬链接到 node_modules/	
14		
15	# 第二次安装：直接使用缓存	
16	<code>bun install express</code>	
17	# ↓ 检测到缓存中已有	
18	# ↓ 直接硬链接，无需下载！	

TIP

即使不使用 Bun 运行时，也可以单独使用 Bun 作为包管理器获得极速体验。

4.2 锁文件

1	# bun.lockb（二进制格式）	
2	# 比 package-lock.json 和 yarn.lock 更小、更快	
3		
4	# 查看锁文件内容	

```
5 bun install --lockfile
6
7 # 生成文本格式的锁文件（用于调试）
8 bun install --generate-lockfile
```

特点：

- 二进制格式：更小、解析更快
- 确定性：确保一致的依赖安装
- 兼容性：可以读取 package-lock.json 和 yarn.lock

NOTE

bun.lockb 是二进制格式，不适合人工阅读，但性能极佳。

4.3 兼容性

兼容 npm：

```
1 # Bun 可以读取 package.json
2 bun install
3
4 # 兼容 npm scripts
5 bun run dev
6 bun run build
7
8 # 兼容 npm registry
9 bun add express # 从 npm registry 下载
```

 Bash

私有 registry：

```
1 # 配置私有 registry
2 bun config set registry https://registry.npmirror.com
3
4 # 或使用 .npmrc
5 # registry=https://registry.npmirror.com
```

 Bash

workspace 支持：

```
1 # Bun 支持 npm workspaces
2 bun install # 自动识别 workspaces
3
4 # 在特定 workspace 中运行
5 bun run --filter=pkg-a build
```

 Bash

4.4 与其他包管理器对比

特性	npm	yarn	pnpm	Bun
安装速度	慢	中	快	极快
磁盘空间	高	高	低	低
锁文件格式	JSON	YAML	YAML	二进制
严格模式	✗	✗	✓	✓

workspace	✓	✓	✓	✓
零安装	✗	✓	✗	✗
生态成熟度	极高	高	中高	中

选择建议:

场景	推荐	原因
新手入门	npm	默认、文档多
追求速度	Bun	最快
节省空间	pnpm	硬链接
monorepo	pnpm/Bun	性能好
企业项目	npm/pnpm	稳定
创新实验	Bun/Yarn PnP	新特性

本章完

Chapter 15

构建工具

Chapter 16

代码质量

Chapter 17

测试框架

Chapter 18

部署与 CI/CD

VIII

JavaScript 底层原理